

Working with CSV Files using Pandas

In this notebook, we will learn how to work with CSV files using Pandas.

CSV files are one of the most commonly used file formats in Machine Learning and Data Science.

We will cover:

- Reading CSV files
- Loading CSV from URL
- Handling separators
- Working with headers
- Selecting columns
- Handling missing values
- Parsing dates
- Reading large files in chunks

```
In [1]: import pandas as pd
```

Reading a CSV File

`pd.read_csv()` is used to load CSV files into a Pandas DataFrame.

```
In [2]: df = pd.read_csv('aug_train.csv')
df.head()
```

```
Out[2]:
```

	enrollee_id	city	city_development_index	gender	relevent_experience	enrolled_u
0	8949	city_103	0.920	Male	Has relevent experience	no_e
1	29725	city_40	0.776	Male	No relevent experience	no_e
2	11561	city_21	0.624	NaN	No relevent experience	Full tir
3	33241	city_115	0.789	NaN	No relevent experience	
4	666	city_162	0.767	Male	Has relevent experience	no_e

```
In [3]: df.shape
```

```
Out[3]: (19158, 14)
```

Reading CSV from URL

Sometimes datasets are stored online.

We can fetch the data using the `requests` library and then load it into Pandas.

```
In [4]: import requests
        from io import StringIO

        url = "https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv"
        headers = {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:66.0)"}
        req = requests.get(url, headers=headers)
        data = StringIO(req.text)

        df = pd.read_csv(data)
        df.head()
```

```
Out[4]:
```

	Country	Region
0	Algeria	AFRICA
1	Angola	AFRICA
2	Benin	AFRICA
3	Botswana	AFRICA
4	Burkina	AFRICA

```
In [5]: df.shape
```

```
Out[5]: (194, 2)
```

Working with Different Separators and Adding Column Names

By default, Pandas expects comma-separated values.

Some files use:

- tabs
- semicolons
- pipes

In such cases, we use the `sep` parameter.

Sometimes datasets do not contain column names.

We can manually provide them using the `names` parameter.

```
In [6]: df = pd.read_csv('movie_titles_metadata.tsv', sep='\t', names=['sno', 'name', 'relea'])
        df.head()
```

Out[6]:

	sno	name	release_year	rating	votes	genres
0	m0	10 things i hate about you	1999	6.9	62847.0	['comedy' 'romance']
1	m1	1492: conquest of paradise	1992	6.2	10421.0	['adventure' 'biography' 'drama' 'history']
2	m2	15 minutes	2001	6.1	25854.0	['action' 'crime' 'drama' 'thriller']
3	m3	2001: a space odyssey	1968	8.4	163227.0	['adventure' 'mystery' 'sci-fi']
4	m4	48 hrs.	1982	6.9	22289.0	['action' 'comedy' 'crime' 'drama' 'thriller']

Setting an Index Column

If a dataset already contains a unique ID column, we can use it as the index.

In [7]: `df = pd.read_csv('aug_train.csv', index_col = 'enrollee_id')`
`df.head()`

Out[7]:

	city	city_development_index	gender	relevent_experience	enrolled_univ
enrollee_id					
8949	city_103	0.920	Male	Has relevent experience	no_enro
29725	city_40	0.776	Male	No relevent experience	no_enro
11561	city_21	0.624	NaN	No relevent experience	Full time c
33241	city_115	0.789	NaN	No relevent experience	
666	city_162	0.767	Male	Has relevent experience	no_enro

Header Parameter

Sometimes the actual column names are not present in the first row.

In such cases, we use the `header` parameter.

In [8]: `df = pd.read_csv('test.csv', header = 1)`
`df.head()`

Out[8]:

	0	enrollee_id	city	city_development_index	gender	relevent_experience	enrollee
0	1	29725	city_40	0.776	Male	No relevent experience	n
1	2	11561	city_21	0.624	NaN	No relevent experience	Fu
2	3	33241	city_115	0.789	NaN	No relevent experience	
3	4	666	city_162	0.767	Male	Has relevent experience	n



Selecting Specific Columns

Real-world datasets may contain many unnecessary columns.

Using `usecols` helps load only required columns.

```
In [9]: df = pd.read_csv('aug_train.csv', usecols = ['enrollee_id', 'gender', 'education_level'])
df.head()
```

Out[9]:

	enrollee_id	gender	education_level
0	8949	Male	Graduate
1	29725	Male	Graduate
2	11561	NaN	Graduate
3	33241	NaN	Graduate
4	666	Male	Masters

Skipping Rows

Sometimes datasets contain unnecessary rows.

We can skip them using `skiprows`.

```
In [10]: df = pd.read_csv('aug_train.csv', skiprows = [0,2])
df.shape
```

Out[10]: (19156, 14)

Loading Limited Rows

Large datasets can consume a lot of memory.

Using `nrows` helps load only a fixed number of rows.

```
In [11]: df = pd.read_csv('aug_train.csv', nrows=100)
df
```

```
Out[11]:
```

	enrollee_id	city	city_development_index	gender	relevent_experience	enrolled
0	8949	city_103	0.920	Male	Has relevent experience	no_
1	29725	city_40	0.776	Male	No relevent experience	no_
2	11561	city_21	0.624	NaN	No relevent experience	Full t
3	33241	city_115	0.789	NaN	No relevent experience	
4	666	city_162	0.767	Male	Has relevent experience	no_
...	...	...	...	...	...	
95	12081	city_65	0.802	Male	Has relevent experience	Full t
96	7364	city_160	0.920	NaN	No relevent experience	Full t
97	11184	city_74	0.579	NaN	No relevent experience	Full t
98	7016	city_65	0.802	Male	Has relevent experience	no_
99	8695	city_11	0.550	Male	Has relevent experience	no_

100 rows × 14 columns



Handling Encoding Issues

Some datasets use different text encodings.

In such cases, we use the `encoding` parameter.

```
In [12]: df = pd.read_csv('zomato.csv', encoding='latin-1')
df.head()
```

Out[12]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Localit Verbos
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century Cit Ma Poblacion Makati City Mak
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo Legaspi Village Makati City Ma
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortiga Mandaluyon City, Ma
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megama Ortiga Mandaluyon City, Mandal
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megama Ortiga Mandaluyon City, Mandal

5 rows × 21 columns



Handling Bad Lines and Dtype Warnings

Some real-world CSV files may contain:

- corrupted rows
- extra separators
- mixed datatypes

In this dataset:

- some rows contain extra columns
- some columns contain mixed datatypes

To handle this:

- `on_bad_lines="skip"` skips corrupted rows
- `low_memory=False` helps Pandas properly detect datatypes
- `encoding="latin-1"` handles special characters

```
In [20]: columns = [
    "ISBN",
    "Book-Title",
    "Book-Author",
    "Year-Of-Publication",
    "Publisher",
    "Image-URL-S",
    "Image-URL-M",
    "Image-URL-L"
]
df = pd.read_csv('BX-Books.csv', sep=';', encoding="latin-1", on_bad_lines="skip")
df.head()
```

```
Out[20]:
```

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	
0	0195153448	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.co
1	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.co
2	0060973129	Decision in Normandy	Carlo D'Este	1991	HarperPerennial	http://images.amazon.co
3	0374157065	Flu: The Story of the Great Influenza Pandemic...	Gina Bari Kolata	1999	Farrar Straus Giroux	http://images.amazon.co
4	0393045218	The Mummies of Urumchi	E. J. W. Barber	1999	W. W. Norton & Company	http://images.amazon.co

Understanding Data Types

Pandas automatically detects data types.

We can manually define them using `dtype`.

```
In [21]: df = pd.read_csv("aug_train.csv", dtype={"target": "int"})
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19158 entries, 0 to 19157
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   enrollee_id                          19158 non-null  int64
1   city                                 19158 non-null  object
2   city_development_index               19158 non-null  float64
3   gender                               14650 non-null  object
4   relevent_experience                  19158 non-null  object
5   enrolled_university                 18772 non-null  object
6   education_level                     18698 non-null  object
7   major_discipline                    16345 non-null  object
8   experience                           19093 non-null  object
9   company_size                        13220 non-null  object
10  company_type                         13018 non-null  object
11  last_new_job                         18735 non-null  object
12  training_hours                       19158 non-null  int64
13  target                              19158 non-null  int64
dtypes: float64(1), int64(3), object(10)
memory usage: 2.0+ MB
```

Parsing Date Columns

Dates are usually loaded as strings.

Using `parse_dates` converts them into datetime format.

```
In [22]: df = pd.read_csv('IPL Matches 2008-2020.csv', parse_dates=['date']).info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 816 entries, 0 to 815
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    816 non-null    int64
1   city                                 803 non-null    object
2   date                                 816 non-null    datetime64[ns]
3   player_of_match                      812 non-null    object
4   venue                                816 non-null    object
5   neutral_venue                       816 non-null    int64
6   team1                                816 non-null    object
7   team2                                816 non-null    object
8   toss_winner                          816 non-null    object
9   toss_decision                        816 non-null    object
10  winner                               812 non-null    object
11  result                               812 non-null    object
12  result_margin                        799 non-null    float64
13  eliminator                           812 non-null    object
14  method                               19 non-null     object
15  umpire1                              816 non-null    object
16  umpire2                              816 non-null    object
dtypes: datetime64[ns](1), float64(1), int64(2), object(13)
memory usage: 108.5+ KB
```

Using Converters

Converters allow us to transform values while loading data.

In [23]: `def rename_team(team):`

```
    if team == "Royal Challengers Bangalore":
        return "RCB"

    return team
```

In [24]: `rename_team("Royal Challengers Bangalore")`

Out[24]: 'RCB'

In [25]: `df = pd.read_csv('IPL Matches 2008-2020.csv', converters={'team1': rename_team})`
`df.head()`

Out[25]:

	id	city	date	player_of_match	venue	neutral_venue	team1
0	335982	Bangalore	2008-04-18	BB McCullum	Chinnaswamy Stadium	0	RCB
1	335983	Chandigarh	2008-04-19	MEK Hussey	Punjab Cricket Association Stadium, Mohali	0	Kings XI Punjab
2	335984	Delhi	2008-04-19	MF Maharoof	Feroz Shah Kotla	0	Delhi Daredevils
3	335985	Mumbai	2008-04-20	MV Boucher	Wankhede Stadium	0	Mumbai Indians
4	335986	Kolkata	2008-04-20	DJ Hussey	Eden Gardens	0	Kolkata Knight Riders

Handling Missing Values

Some datasets contain custom missing value placeholders like:

- "-"
- "missing"
- "not available"

We can convert them into NaN values using `na_values`.

In [30]: `df = pd.read_csv('aug_train.csv', na_values=['Male',])`
`df.head()`

Out[30]:

	enrollee_id	city	city_development_index	gender	relevent_experience	enrolled_u
0	8949	city_103	0.920	NaN	Has relevent experience	no_e
1	29725	city_40	0.776	NaN	No relevent experience	no_e
2	11561	city_21	0.624	NaN	No relevent experience	Full tir
3	33241	city_115	0.789	NaN	No relevent experience	
4	666	city_162	0.767	NaN	Has relevent experience	no_e



Reading Large Datasets in Chunks

Large datasets may not fit completely into memory.

Using `chunksize` helps process the data in smaller parts.

```
In [40]: dfs = pd.read_csv('aug_train.csv', chunksize=5000)
```

```
In [41]: for chunk in dfs:
          print(chunk.shape)
```

```
(5000, 14)
```

```
(5000, 14)
```

```
(5000, 14)
```

```
(4158, 14)
```

```
In [ ]:
```